Lab3

STAT 4610-5610: Spring 2026

2026-03-23

Lab3: Support Vector Machines

Due: 11:59PM Friday, March 27 on Canvas as a knitted pdf file of your team's Quarto document

1. You will individually fit a support vector machine, describe what it is doing, and report its misclassification rate, sensitivity, and specificity. (I will do a linear SVM)
2. You will interpret this model and make a prediction in your individual section.
3. As a team, you will create a support vector machine (SVM) and compete against other teams for the best test performance. You will also create a team plot that summarizes the performance of the individual SVMs and your team's SVM.

Instructions for Lab3

In Lab0 (and Lab1), the professor created a generating model for Y variables taking on 0 or 1 values based on the X1 and X2 inputs. You came up with the backstory for what Y, X1, and X2 were and why it was important to use X1 and X2 to predict Y.

In this lab, you will apply support vector machines to continue your analyses for how to best predict Y from X1 and X2. You should continue to use the Q1 qualitative context you developed in Lab0 or Lab1 (or you can develop a new one if you want).

Each individual will fit at least one SVM on a training dataset and then evaluate how well the SVM classifies Y based on a test dataset. Individuals will make a prediction given (x1, x2) and then interpret their prediction and make a recommendation for action. Unlike in Lab0 and Lab1, you will not describe the whole workflow of Q1, Q2, and Q3 for this "project". You will report on the misclassification rate, sensitivity, and specificity for your SVM and make a prediction given (x1, x2). You will comment on how the prediction differs from your prediction in Lab1 and, given this new prediction, describe what actions (Q3) you recommend given your Q2 results.

Generating Model

We have a logistic regression generating model. Given $x_1 \in [0,1]$ and $x_2 \in [0,1], Y \sim Ber(p)$, where p is related to x_1 and x_2 through the logistic link function: $\log(\frac{p}{(1-p)}) = x_1 - 2x_2 - 2{x_1}^2 + {x_2}^2 + 3x_1x_2$, where log is the natural log, sometimes written as ln.

The code for this is below.

```
library(class)
suppressPackageStartupMessages(library(tidyverse))
```

```
#Generative model
set.seed(200) #setting a random seed so that we can
#reproduce everything exactly if we want to

generate_y <- function(x1,x2) { #two input parameters to generate the output y
  logit <- x1 -2*x2 -2*x1^2 + x2^2 + 3*x1*x2
  p <- exp(logit)/(1+exp(logit)) #apply the inverse logit function
  y <- rbinom(1,1,p) #y becomes a 0 (with prob 1-p) or a 1 with probability p
}
```

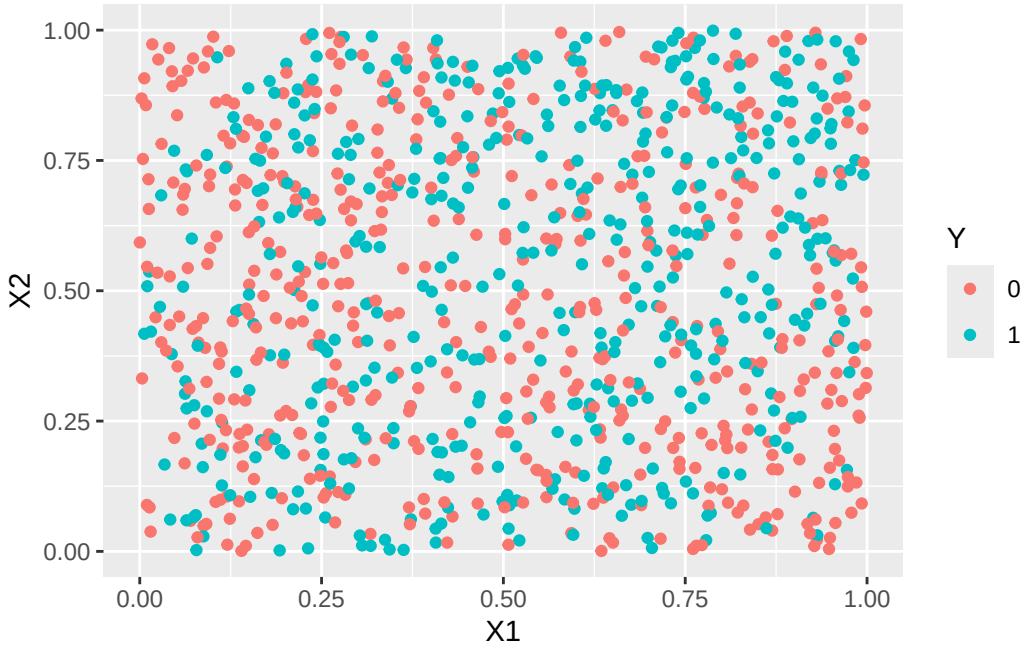
Training dataset

We are going to use our generating model to create a training dataset of 1000 predictors (x1, x2), and then 1000 outcomes. Then we plot all three variables to see what our training data looks like.

```
# Generate a training dataset with 1000 points
set.seed(321)
n = 1000
X1 <- runif(n,0,1)
X2 <- runif(n,0,1)

#I'm going to use a for loop to generate 1000 y's
Y <- rep(0,n) #initializing my Y to be a vector of 0's
for (i in 1:n) {
  Y[i] <- generate_y(X1[i],X2[i])
}

sum(Y) #How many 0's and 1's were predicted? When n is very large
```

```
[1] 465
```

```
# about 48% of the Y's should be 1's.

training <- as.data.frame(cbind(X1, X2, Y))
training$Y <- as.factor(training$Y) #combining all of my variables into a training dataset
ggplot(data=training, aes(x=X1, y=X2, color=Y)) +
  geom_point()
```



How well will various SVMs predict Y given new x1 and x2 values?

Test datasets

Each individual will generate a test set of 100 predictors (x1, x2) and outcomes (y) that we will use as our "ground truth". The test dataset should be the same for each team member. Use seed=323.

```
# Create the training dataset as above using seed=321
# Create a testing dataset using seed=323

set.seed(323)
n = 100

X1_test <- runif(n,0,1)
X2_test <- runif(n,0,1)
```

```r
Y_test <- rep(0,n)
for (i in 1:n) {
  Y_test[i] <- generate_y(X1_test[i],X2_test[i])
}

test <- as.data.frame(cbind(X1_test,X2_test,Y_test))

table(test$Y_test)
```

```
 0  1
51 49
```

What individuals need to do (20 pts)

1. Given the training set (seed=321), fit a ***different*** SVM for each team member. Choose from the options below:
    1. linear SVM (this will be a support vector classifier, i.e., a flat hyperplane [actually just a line])
    2. SVM with an interaction between X1 and X2
    3. SVM with degree 2 polynomials
    4. SVM with degree 3 polynomials
2. Use cross validation to choose your tuning parameters.
3. Describe what the SVM is doing, i.e., how it classifies points.
4. Calculate the sensitivity, specificity, and overall error rate (misclassification rate) for your SVM given the test set of 100 data points.
5. Make a prediction for a new point (x1, x2) = (0.25, 0.25) or (0.25, 0.75) or (0.75, 0.25) or (0.75, 0.75) for each fitted model. Each individual will have a different point for their predictions. Use the same one you used for Lab1. I will use (.25, .25)
6. Comment on how the prediction differs from your prediction(s) in Lab1 and, given this new prediction, describe what actions (Q3) you recommend given your Q2 results.

Emery Individual Section

```r
library(e1071)
```

```
Attaching package: 'e1071'
```

```
The following object is masked from 'package:ggplot2':

    element
```

Fit Linear SVM on training data using CV (Using tune() to perform 10-fold CV)

```r
tune_linear <- tune(svm, Y ~ X1 + X2,
                    data = training,
                    kernel = "linear",
                    ranges = list(cost = c(0.01, 0.1, 1, 10, 100)))
summary(tune_linear)
```

```
Parameter tuning of 'svm':

- sampling method: 10-fold cross validation

- best parameters:
 cost
    1

- best performance: 0.455

- Detailed performance results:
   cost error dispersion
1 1e-02 0.463 0.05850926
2 1e-01 0.465 0.05104464
3 1e+00 0.455 0.05016639
4 1e+01 0.456 0.04835057
5 1e+02 0.456 0.04835057
```

```r
best_linear <- tune_linear$best.model
```

A linear SVM classifies points by finding the optimal hyperplane (in this case a line since we have only two predictors) that best separates the two classes ($Y = 0$ and $Y = 1$). "Optimal" means the hyperplane that maximizes the margin which is the distance between the hyperplane and the closest data points from each class. These points are known as support vectors. Only these support vectors influence the position of the decision boundary, the remaining training points are irrelevant to it.

The cost parameter C, tuned via 10-fold cross-validation, controls the trade-off between maximizing the margin and minimizing misclassification. A larger C allows for more misclassifications

in exchange for a wider margin, while a smaller C penalizes misclassifications heavily, resulting in a narrower margin that tries to correctly classify more training points.

Once trained, the model classifies a new point by determining which side of the hyperplane it falls on. Points on one side are predicted Y = 0, and points on the other side are predicted Y = 1.

Predict on The Test Set

```
test <- as.data.frame(cbind(X1_test, X2_test, Y_test))
names(test) <- c("X1", "X2", "Y")
test$Y <- as.factor(test$Y)

pred_linear <- predict(best_linear, newdata = test)
```

Confusion Matrix for Sensitivity and Specificity

```
test$Y <- factor(test$Y, levels = c("0", "1"))
pred_linear <- factor(pred_linear, levels = c("0", "1"))

conf <- table(Predicted = pred_linear, Actual = test$Y)
conf
```

```
         Actual
Predicted  0  1
        0 35 29
        1 16 20
```

```
sensitivity <- conf[2,2] / sum(conf[,2])
specificity <- conf[1,1] / sum(conf[,1])
error_rate  <- mean(pred_linear != test$Y)

sensitivity
```

```
[1] 0.4081633
```

```
specificity
```

```
[1] 0.6862745
```

```r
error_rate
```

```
[1] 0.45
```

New Predicition

```r
new_point <- data.frame(X1 = 0.25, X2 = 0.25)

prediction <- predict(best_linear, newdata = new_point)
actual <- generate_y(0.25, 0.25)

cat("For the new point (X1 = 0.25, X2 = 0.25):\n",
    "Predicted Y =", as.character(prediction), "\n",
    "Actual Y    =", actual, "\n",
    "Correct?    =", ifelse(as.character(prediction) == as.character(actual), "Yes", "No"))
```

```
For the new point (X1 = 0.25, X2 = 0.25):
 Predicted Y = 0
 Actual Y    = 1
 Correct?    = No
```

Lab 1 Comparison and Q3 Recommendation

The linear SVM predicted Y = 0 for the new point (X1 = 0.25, X2 = 0.25), however the true value was Y = 1, meaning the model misclassified this point. This is consistent with Lab 1, where logistic regression, LDA, QDA, and Naive Bayes all also predicted Y = 0. Looking back, we can actually see that the linear SVM only predicts 0, likely because there is too much overlap in the data, and a Linear SVM simply cannot do a good job separating the classes. Notably, KNN with optimal K was the only method across both labs to correctly classify this point, likely because KNN classifies based on the local neighborhood of nearest neighbors rather than a global decision boundary.

In Lab 1, I proposed possible context for this dataset: X1 represents the average summer temperature of a region and X2 represents the average summer rainfall. The goal was to predict whether a region experiences significant wildfire damage (Y = 1) or not (Y = 0). reflecting on this goal in Q3, the false negative and constant prediction of 0 is concerning for wildfire damage prediction. It means many regions that may truly need wildfire assistance are being overlooked. I recommend trying more flexible, locally sensitive classification approaches over a linear boundary when predicting wildfire risk. More flexible SVM approaches, such as polynomial or radial basis function (RBF) kernel SVMs, may better capture the non-linear relationship between average summer temperature, rainfall, and wildfire damage. Using higher

degree SVMs could improve classification accuracy, ultimately reducing the risk of missing regions that are truly at high risk for significant wildfire damage.

What teams need to do

1. Create a new team SVM that predicts the test set better than any individual SVM.
2. Describe what the SVM is doing, i.e., how it classifies points.
3. Make a plot that summarizes the performance of the individual SVMs and your team's SVM.
4. Provide R code that can easily be run in class to test the performance of your team's SVM (on a new test set). The code must accept a test dataset of 100 X and y values and output the misclassification rate, sensitivity, and specificity of the SVM. More specifically, write the code so that the only thing that needs to change is the random seed. Add some aspect of your team name to your code so we can distinguish between different teams' functions. Email your .qmd file to the professor. (Upload a pdf to Canvas.) The team with the best classifier on the test set the professor will generate in class (using a different seed) will get 10 bonus points. Second place will get 5 bonus points. Third place will get 4 bonus points. Fourth place will get 3 bonus points. Fifth place will get 2 bonus points. Sixth place will get 1 bonus point. Only teams whose code runs (in class) on the first try will get any bonus points.
5. In the team section of this lab, write a paragraph about individual contributions. Each individual must comment on their contributions to the lab. Only individuals who contribute to the team section will get points for the team section.

Some intended outcomes from this assignment:

- You will individually get practice fitting a SVM
- You will compare the predictive performance of multiple SVMs and previously used classification methods
- You will make predictions based on your SVM
- You will compare models visually
- You will gain experience collaborating with your teammates on an applied problem